MASM Assembly Environment Setup — Summary

A reference guide for configuring Visual Studio 2026 from scratch to assemble, link, and debug x86 MASM programs, based on the setup process completed step by step.

1. Prerequisites

- **Visual Studio 2026** installed with the **Desktop development with C++** workload (this bundles `ml.exe`, the Microsoft Macro Assembler).
- **Irvine32 library files** downloaded and extracted to `C:\Irvine` (do not rename or relocate this folder — sample projects and includes assume this exact path).

2. Creating a New Project From Scratch

1. **File > New > Project** → select **Empty Project** under the C++ category (not "Console App", which adds unwanted C++ starter files).
2. Name the project and choose a location, then click **Create**.

3. Enabling MASM Support

Visual Studio doesn't assemble `.asm` files by default — it needs to be told to use MASM instead of the C++ compiler.

1. Right-click the project in **Solution Explorer** → **Build Customizations...**
2. Check the box for `masm(.targets, .props)`.
3. Click **OK**.

4. Adding the Source File

1. Right-click the project → **Add** → **New Item**.
2. Choose any template (e.g. C++ File) but rename it with a `.asm` extension, e.g. `MyProgram.asm`.
3. Click **Add**, then paste in your assembly source code.

4. Visual Studio automatically detects `.asm` files as **Microsoft Macro Assembler** items once the build customization is enabled — no manual "Item Type" change needed.

5. Platform Configuration

- Set the **Platform** dropdown (top toolbar, or in **Configuration Manager**) to **Win32** — not x64.
- 32-bit MASM code (`.386` directive, `eax`/`ebx` registers, `stdcall` calling convention) requires the Win32 platform to build and run correctly.

6. Configuring Include and Library Paths

Only required if your program uses `INCLUDE Irvine32.inc` or calls Irvine procedures (e.g. `DumpRegs`, `WriteInt`). Not needed for bare Windows API-only programs.

Setting	Where	Value
Assembler Include Paths	Right-click `.asm` file → Properties → Microsoft Macro Assembler → Include Paths	`C:\Irvine`
Library Directories	Project → Properties → VC++ Directories → Library Directories	`C:\Irvine`
Additional Dependencies	Project → Properties → Linker → Input → Additional Dependencies	`Irvine32.lib;kernel32.lib;%(AdditionalDependencies)`

For programs that only call `ExitProcess` directly (no Irvine procedures), `kernel32.lib` alone is sufficient and the Include/Library Directory settings can be skipped.

7. Building and Running

1. **Build > Build Project** (or **Rebuild Project** if no changes are detected).
2. Check the **Output** window for a success message with no errors.
3. Set a breakpoint by clicking the margin next to an executable line (a red dot appears).
4. **Debug > Start Debugging** (F5) — execution pauses at the breakpoint.
5. Open **Debug > Windows > Registers** to watch register values.
6. Use **F10 (Step Over)** to execute one instruction at a time and observe register changes (e.g. `EAX` updating after a `mov` or `add`).

8. Antivirus False Positives (Common Issue)

Freshly assembled `.exe` files — especially ones linking `Irvine32.lib` and calling its procedures — are frequently flagged by Windows Defender as a false positive. This happens because:

- Irvine procedures call low-level Windows console/API functions directly, a pattern heuristic scanners associate with certain malware behavior.
- Static linking increases binary size/complexity compared to a bare `ExitProcess`-only program.
- The executable is unsigned, as with any local student build.

Fix:

1. Open **Windows Security > Virus & threat protection > Protection history**.
2. Find the quarantined `.exe`, click it, and choose **Actions > Restore**.
3. Go to **Manage settings > Exclusions > Add or remove exclusions > Add an exclusion > Folder**.
4. Add your project's `Debug` folder (e.g. `C:\...\YourProject\Debug`) to the exclusion list.
5. **Rebuild** the project in Visual Studio, then run/debug again — Defender will no longer intercept it.

Quick Reference: Bare vs. Irvine32 Programs

	Bare `ExitProcess` program	**Irvine32 (`DumpRegs`, etc.) program**
MASM build customization	Required	Required
Platform	Win32	Win32
Assembler Include Paths	Not needed	`C:\Irvine`
Library Directories	Not needed	`C:\Irvine`
Additional Dependencies	`kernel32.lib`	`Irvine32.lib;kernel32.lib`
Console output	None — check via Registers window	Visible directly via `call DumpRegs`

	Bare `ExitProcess` program	**Irvine32 (`DumpRegs`, etc.) program**
Antivirus false positives	Less likely	More likely